

# DSA Project Report

Team Name: cul-de-sacs

Course: CS 231

Sanaa Tabassum (24b1078), Jaspreet Brar (24b1082), Medhansh Verma (24b09xx)

November 22, 2025

## Contents

|          |                                            |           |
|----------|--------------------------------------------|-----------|
| <b>1</b> | <b>Directory Structure of Code</b>         | <b>3</b>  |
| <b>2</b> | <b>Makefile and Targets</b>                | <b>4</b>  |
| <b>3</b> | <b>Assumptions</b>                         | <b>7</b>  |
| <b>4</b> | <b>Test Cases and Analysis</b>             | <b>7</b>  |
| <b>5</b> | <b>Time and Space Complexity</b>           | <b>8</b>  |
| 5.1      | Phase 1 . . . . .                          | 8         |
| 5.1.1    | Shortest Path . . . . .                    | 8         |
| 5.1.2    | KNN . . . . .                              | 8         |
| 5.2      | Phase 2 . . . . .                          | 9         |
| 5.2.1    | K shortest Paths (Exact) . . . . .         | 9         |
| 5.2.2    | K Shortest Paths (Heuristic) . . . . .     | 10        |
| 5.2.3    | Approximate Shortest Paths . . . . .       | 11        |
| <b>6</b> | <b>Approach for Each Query Type</b>        | <b>11</b> |
| 6.1      | Phase 1 . . . . .                          | 11        |
| 6.1.1    | Shortest Path . . . . .                    | 11        |
| 6.1.2    | KNN . . . . .                              | 12        |
| 6.2      | Phase 2 . . . . .                          | 12        |
| 6.2.1    | K shortest Paths (Exact) . . . . .         | 12        |
| 6.2.2    | K Shortest Paths (Heuristic) . . . . .     | 13        |
| 6.2.3    | Approximate Shortest Paths . . . . .       | 13        |
| <b>7</b> | <b>Phase 3: Delivery Scheduling</b>        | <b>14</b> |
| 7.1      | Our Initial Plan . . . . .                 | 14        |
| 7.2      | Attempt 1: Time-Based Dijkstra . . . . .   | 14        |
| 7.3      | Attempt 2: Using Landmarks (ALT) . . . . . | 14        |
| 7.4      | Attempt 3: Greedy Nearest Task . . . . .   | 15        |
| 7.5      | Final Working Solution . . . . .           | 15        |
| 7.6      | Trade-offs . . . . .                       | 16        |

|                          |           |
|--------------------------|-----------|
| 7.7 Conclusion . . . . . | 16        |
| <b>8 AI Use</b>          | <b>16</b> |

# 1 Directory Structure of Code

Provide the structure of your project. Example:

```
cul-de-sacs/
Driver1.cpp          % Driver for Phase 1 queries
Driver2.cpp          % Driver for Phase 2 queries
Driver3.cpp          % Driver for Phase 3 queries
Makefile             % Build automation for all phases

Phase-1/             % Phase 1: Basic shortest paths + KNN
  graph.cpp          % Graph class implementation
  graph.h            % Graph class header
  knn.cpp            % K-Nearest Neighbour implementation
  query.h            % Query structures for Phase 1
  shortest-path.cpp  % shortest-path module
  graph.o            % Compiled object file
  knn.o              % Compiled object file
  shortest-path.o    % Compiled object file

Phase-2/             % Phase 2: K-shortest paths + Approx. paths
  approximate-shortest-paths.cpp % ALT heuristic approximate shortest paths
  k-shortest-exact.cpp % Exact Yen-style K-shortest paths
  k-shortest-heurestics.cpp % Heuristic K-shortest variation
  graph.cpp          % Graph implementation for Phase 2
  graph.h            % Graph header
  shortest-path.cpp  % Re used from Phase 1
  query.h            % Query structures for Phase 2
  approximate-shortest-paths.o % Compiled object file
  k-shortest-exact.o % Compiled object file
  k-shortest-heurestics.o % Compiled object file
  shortest-path.o    % Compiled object file

Phase-3/             % Phase 3: Scheduler + Dynamic Simulations
  graph.cpp          % Graph implementation for Phase 3
  graph.h            % Graph header for scheduling queries
  scheduler.cpp       % Event scheduler (dynamic updates)
  scheduler.h         % Scheduler header
  json.h             % Lightweight JSON reader for Phase 3
  graph.o            % Compiled object file
  scheduler.o         % Compiled object file

Report.pdf           % Final report for submission
main.cpp            % Test harness / local runner

compare_t1.py        % Comparison tool for Phase 1 tests
compare_t2.py        % Comparison tool for Phase 2 tests
gen_tests_phase1.py  % Generator for Phase 1 test cases
```

```

gen_tests_t1.py          % Generator for grading tests (type 1)
gen_k_heurestics.py      % Generator for heuristic K-shortest tests

nlohmann/                % JSON library used in all phases
    json.hpp
    json_fwd.hpp

```

Write the purpose of each folder and key file.

## 2 Makefile and Targets

Our Makefile:

```

# =====
# Graph-Based Routing System Simulation
# Makefile for Phase 1, Phase 2, and Phase 3
# =====

# Compiler and Flags
CXX := g++
CXXFLAGS := -std=c++17 -O2 -Wall -Wextra -Wno-unused-variable -Wno-unused-parameter

# Directories
PHASE1_DIR := Phase-1
PHASE2_DIR := Phase-2
PHASE3_DIR := Phase-3

# Executables
PHASE1_EXE := phase1
PHASE2_EXE := phase2
PHASE3_EXE := phase3

# Common Source (e.g., driver)
DRIVER1 := Driver1.cpp
DRIVER2 := Driver2.cpp
DRIVER3 := Driver3.cpp

# Collect source files from each phase
PHASE1_SRCS := $(wildcard $(PHASE1_DIR)/*.cpp)
PHASE2_SRCS := $(wildcard $(PHASE2_DIR)/*.cpp)
PHASE3_SRCS := $(wildcard $(PHASE3_DIR)/*.cpp)

# Object files (build artifacts)
PHASE1_OBJS := $(PHASE1_SRCS:.cpp=.o)
PHASE2_OBJS := $(PHASE2_SRCS:.cpp=.o)
PHASE3_OBJS := $(PHASE3_SRCS:.cpp=.o)

```

```

# =====
# Targets
# =====

all: $(PHASE1_EXE) $(PHASE2_EXE) $(PHASE3_EXE)

# -----
# Phase 1 Build
# -----
$(PHASE1_EXE): $(PHASE1_OBJS) $(DRIVER1)
    @echo " Building Phase 1 executable..."
    $(CXX) $(CXXFLAGS) -o $@ $(DRIVER1) $(PHASE1_OBJS)
    @echo " Built ./$$(PHASE1_EXE)"

# -----
# Phase 2 Build
# -----
$(PHASE2_EXE): $(PHASE2_OBJS) $(DRIVER2)
    @echo " Building Phase 2 executable..."
    $(CXX) $(CXXFLAGS) -o $@ $(DRIVER2) $(PHASE2_OBJS)
    @echo " Built ./$$(PHASE2_EXE)"

# -----
# Phase 3 Build
# -----
$(PHASE3_EXE): $(PHASE3_OBJS) $(DRIVER3)
    @echo " Building Phase 3 executable..."
    $(CXX) $(CXXFLAGS) -o $@ $(DRIVER3) $(PHASE3_OBJS)
    @echo " Built ./$$(PHASE3_EXE)"

# -----
# Run Commands (for convenience)
# -----
run-phase1:
    @echo " Running Phase 1..."
    ./$$(PHASE1_EXE) graph.json queries.json output.json

run-phase2:
    @echo " Running Phase 2..."
    ./$$(PHASE2_EXE) graph.json queries.json output.json

run-phase3:
    @echo " Running Phase 3..."
    ./$$(PHASE3_EXE) graph.json queries.json output.json

# -----
# Utility Targets
# -----

```

```

clean:
    @echo " Cleaning object files..."
    rm -f $(PHASE1_DIR)/*.o $(PHASE2_DIR)/*.o $(PHASE3_DIR)/*.o

distclean: clean
    @echo " Removing executables..."
    rm -f $(PHASE1_EXE) $(PHASE2_EXE) $(PHASE3_EXE)

help:
    @echo "Available targets:"
    @echo "  make all           - Build all phases"
    @echo "  make phase1        - Build phase 1 only"
    @echo "  make phase2        - Build phase 2 only"
    @echo "  make phase3        - Build phase 3 only"
    @echo "  make run-phase1    - Run phase1 with sample input/output"
    @echo "  make run-phase2    - Run phase2 with sample input/output"
    @echo "  make run-phase3    - Run phase3 with sample input/output"
    @echo "  make clean         - Remove .o files"
    @echo "  make distclean     - Remove .o files and executables"

# -----
# Phony targets
# -----
.PHONY: all clean distclean run-phase1 run-phase2 run-phase3 help

```

## Explanation of Makefile Targets

- **all**: The default target that builds all three phases of the project. It compiles the code inside each phase directory and produces the executables `phase1`, `phase2`, and `phase3`.  
*Usage*: `make all`
- **phase1**: Compiles all C++ source files inside the `Phase-1/` directory and links them with `Driver1.cpp` to generate the `phase1` executable.  
*Usage*: `make phase1`
- **phase2**: Compiles all C++ source files inside `Phase-2/` and links them with `Driver2.cpp`, producing the `phase2` executable.  
*Usage*: `make phase2`
- **phase3**: Compiles source files inside `Phase-3/` along with `Driver3.cpp` and generates the `phase3` executable.  
*Usage*: `make phase3`
- **run-phase1**: Runs the Phase 1 executable using the expected input format. Internally executes:  
`./phase1 graph.json queries.json output.json`.  
*Usage*: `make run-phase1`
- **run-phase2**: Runs the Phase 2 executable with the same default input files. Useful for quickly testing Phase 2 output.  
*Usage*: `make run-phase2`

- **run-phase3**: Runs the Phase 3 binary with the required JSON files as command-line arguments.  
*Usage: make run-phase3*
- **clean**: Removes all intermediate object files (\*.o) from every phase directory. This is helpful when recompiling from scratch.  
*Usage: make clean*
- **distclean**: Performs everything done by **clean** and also deletes all generated executables (**phase1**, **phase2**, **phase3**). Effectively resets the build output.  
*Usage: make distclean*
- **help**: Prints a list of all available Makefile targets along with a short description of each, making it easy to recall build and run commands.  
*Usage: make help*

### 3 Assumptions

None, we just followed the guidelines file and piazza.

### 4 Test Cases and Analysis

Describe custom test cases.

- `gen_k_heurestics.py` – generates testcases for  $k$ -shortest paths using heuristics.
- `compare_t1.py` – visualises the exact  $k$ -shortest paths on a map.
- `gen_tests_t1.py` – generates testcases for exact  $k$ -shortest paths.
- `compare_t2.py` – visualises the heuristic  $k$ -shortest paths on a map.
- `gen_tests_phase1.py` – generates Phase 1 testcases.

#### Python Libraries Used:

- `json` – reading and writing JSON data.
- `random` – generating random values for testcase variety.
- `math` – mathematical functions (e.g., distances, angles).
- `argparse` – command-line argument parsing.
- `os` – file and directory handling.
- `osmnx` – downloading and working with OpenStreetMap road networks.
- `networkx` – graph algorithms and shortest path computations.

## 5 Time and Space Complexity

### 5.1 Phase 1

#### 5.1.1 Shortest Path

**Time Complexity:** For distance-based queries, the algorithm uses an A\* search where each relaxation step requires examining the outgoing edges of a node. In the worst case, the heuristic does not shorten any part of the search and the complexity matches that of Dijkstra’s algorithm. Using a binary heap priority queue, the running time is:

$$O(E \log N),$$

where  $N$  is the number of vertices and  $E$  is the number of edges.

For time-based queries, the algorithm performs a time-dependent Dijkstra computation. Each relaxation evaluates the travel time of an edge using `calc_edge_time`, which runs in  $O(S)$  time where  $S$  is the number of speed slots (a small constant,  $S = 96$  for 15-minute intervals). Since  $S$  is constant, the asymptotic complexity remains:

$$O(E \log N).$$

Both variants may receive forbidden node and forbidden road-type constraints; these checks occur during relaxation and do not change the overall asymptotic running time. Therefore, the worst-case time complexity for processing a single shortest-path query is:

$$O(E \log N).$$

**Space Complexity:** The algorithms store a distance (or time) array of size  $N$ , a parent array of size  $N$ , and a priority queue that in the worst case may hold up to  $N$  entries. In addition, the implementation maintains small auxiliary arrays for forbidden nodes and road-type masks, all of which are  $O(N)$  or smaller. No additional data structures grow with the size of the graph.

Thus, the overall space usage for each shortest-path computation is:

$$O(N).$$

#### 5.1.2 KNN

**Euclidean Metric:** For the Euclidean distance mode, the algorithm first scans all nodes to find the nearest graph node to the query point and then again to collect all POI candidates. For each candidate, the Euclidean distance is computed and a max-heap of size  $k$  is maintained to store the  $k$  nearest neighbors. The total time complexity is:

$$O(N) + O(N) + O(C) + O(C \log k) = O(N + C \log k).$$

The space complexity comes from storing the POI candidate list and the heap:

$$O(C + k).$$

**Shortest-Path Metric:** In the shortest-path mode, the algorithm utilises `shortest_path_dist_vec`, which performs Dijkstra's algorithm from the nearest graph node. This dominates the running time. Including heap maintenance for selecting the  $k$  nearest POI nodes, the total time complexity is:

$$O(N) + O(N) + O(E \log N) + O(C \log k) = O(M \log N + C \log k).$$

The space use for the Dijkstra distance array, the POI candidate list, and the  $k$ -sized heap:

$$O(N + C + k).$$

where:

N= Number of Nodes

E= Number of Edges

C= Number of Candidates

## 5.2 Phase 2

### 5.2.1 K shortest Paths (Exact)

**Time Complexity:** The algorithm first computes the shortest path from the source to the target using A\*, which takes:

$$O(E \log N),$$

where  $N$  is the number of nodes and  $E$  is the number of edges in the graph.

To generate each additional path, the algorithm considers every vertex on the previously found best path as a spur node. If the average path length is suppose  $L$ , then there are  $L$  spur points. For each spur point, edges are temporarily removed and a new shortest path is computed, again costing:

$$O(E \log N).$$

Since this process is repeated for  $k - 1$  paths(until we find the  $k^{th}$  path) and we do it for around  $L$  spur points, the total worst-case running time is:

$$O((k - 1) L E \log N).$$

**Space Complexity:** Each shortest-path computation A\* requires a distance array and a priority queue, which together use

$$O(N)$$

space.

The algorithm stores the  $k$  final simple paths, each of average length  $L$ , contributing

$$O(kL).$$

During the generation of the remaining  $k - 1$  paths, the algorithm creates candidate paths inside the priority queue. For each of the  $k - 1$  iterations, up to  $L$  spur nodes may be examined, and therefore up to  $O(kL)$  candidate paths may be generated overall. Because

each candidate path is stored as a full vector of up to  $L$  nodes in this implementation, the total storage required for all candidate paths is

$$O(kL^2).$$

The hash set used to prevent duplicates stores one entry per generated path, up to  $O(kL)$  entries. This is asymptotically smaller than  $O(kL^2)$  and does not affect the overall bound.

Likewise, temporary spur paths are immediately incorporated into candidate paths and therefore do not contribute additional asymptotic space apart from the  $O(kL^2)$  term already counted.

Combining all components, the overall worst-case space complexity is:

$$O(N + kL^2).$$

### 5.2.2 K Shortest Paths (Heuristic)

Let  $N$  be the number of vertices,  $E$  the number of edges,  $L$  the length (number of vertices) of the base (shortest) path,  $k$  the requested number of paths (here 2–7), and  $P$  the candidate-cap parameter (`psblts_cap`, e.g. 800). Constants such as  $B$  (max consecutive removals), `spurinterval` and `itr_nodes` are considered small constants.

**Time Complexity:** The algorithm performs one exact shortest-path computation to obtain the base path, costing

$$O(E \log N).$$

It then samples the base path and performs multiple spur computations; up to  $O(L)$  spur-attempts may be generated during sampling, and further local spurs may be generated for each selected candidate. Each spur attempt calls the shortest-path function, which is  $O(E \log N)$  in the worst case. The algorithm also sorts the candidate pool of size at most  $P$ , which costs  $O(P \log P)$ . Therefore a compact worst-case bound, expressed in terms of the candidate cap  $P$ , is:

$$O(P \cdot E \log N + P \log P),$$

which, when sampling dominates and  $P$  is not reached, can be written as:

$$O(L \cdot E \log N + P \log P).$$

Because  $k, B, \text{spurinterval}, \text{itr\_nodes}$  are small constants, they affect only constant factors and not the asymptotic class.

**Space Complexity:** A\* workspace (distance array and PQ) requires  $O(N)$  space. The candidate list `psblts` can hold up to  $P$  candidates and each candidate stores a full path of up to  $L$  nodes, so candidate storage requires:

$$O(P \cdot L).$$

The `psblts_seen` hash table stores up to  $P$  hashes ( $O(P)$ ), and the final selected paths require  $O(kL)$ . Temporary buffers (removed-edge lists, spur-path temporaries) require  $O(L)$  or  $O(1)$  transient space. Combining these terms yields:

$$O(N + P \cdot L).$$

If  $P$  is a fixed constant (e.g. 800), the practical memory usage reduces to  $O(N + L)$ .

### 5.2.3 Approximate Shortest Paths

**Time Complexity:** The preprocessing phase selects  $L$  landmarks using the avoid method. For each chosen landmark, the algorithm performs one forward and one reverse Dijkstra run. Using a binary heap, each run costs

$$O(E \log N),$$

so preprocessing requires:

$$O(L E \log N).$$

For each approximate shortest-path query, the algorithm performs an A\* search using the ALT heuristic. The cost of exploring neighbors remains  $O(E \log N)$  in the worst case, but the landmark-based heuristic substantially reduces the number of nodes expanded in practice. Computing the heuristic at each relaxation requires evaluating up to  $L$  landmark constraints, each in constant time. Thus the heuristic calculation contributes  $O(L)$  per expanded node but does not change the asymptotic bound.

Therefore, the worst case running time per query will be:

$$O(E \log N + L) = O(E \log N).$$

When many queries are processed as a batch, the total running time is linear in the number of queries, and preprocessing is performed only once.

**Space Complexity:** The preprocessing stage stores two distance tables for each of the  $L$  landmarks: `dist_from_landmark` and `dist_to_landmark`, each of size  $N$ . These tables require:

$$O(LN)$$

space.

During an approximate shortest-path query, the algorithm maintains arrays for `g_score`, a visited marker, and a priority queue of up to  $N$  entries, all of which require  $O(N)$  space. No additional structures grow with the size of the graph.

Thus, the overall space complexity of the module is:

$$O(LN).$$

## 6 Approach for Each Query Type

### 6.1 Phase 1

#### 6.1.1 Shortest Path

The shortest path module supports two modes: minimizing distance and minimizing time. For the distance mode, we use an A\*-based approach where the standard Dijkstra cost is combined with a Euclidean heuristic. The heuristic is simply the straight-line distance between a node and the target, which keeps the search directed toward the destination and will thus speed up the exploration. The priority queue will store  $(g + h)$  for each node, where  $g$  is the distance from the source so far, and  $h$  is the estimated remaining distance. The search stops as soon as the target is popped from the queue, and the path is reconstructed from the parent pointers.

For the second mode, minimizing travel time, the algorithm switches to a time-dependent Dijkstra. Each edge can have a varying speed throughout the day, so instead of adding a fixed weight, we compute the actual travel time for each segment based on the current arrival time. The function `calc_edge_time()` handles this by checking which 15-minute speed slot the user is currently in and advancing through the day as needed. This gives a realistic travel-time estimate when traffic speeds change over time.

Both algorithms respect constraints. If the query specifies forbidden nodes or forbidden road types, those are skipped during the relaxation step. An invalid constraint, such as forbidding the source or target, immediately makes the path impossible.

### 6.1.2 KNN

For this query, we want to return  $k$  nearest nodes of the POI type required with respect to the latitude longitude coordinate given. We can use any of the two metrics- euclidean distance or shortest-path distance.

For both the metrics our algorithm identifies the node closest to our query's coordinate using euclidean distance.

We then find all nodes whose POI type matches with the one we want and collect it into a candidate list. If the size of this list is less than  $k$ , we return this list, or else we use our metric.

If the metric is euclidean, we compute the euclidean distance and use a max heap and maintain it with a size of at most  $k$ . Each time we push a candidate into this heap we check if the distance of that element is more than the top most element's. We remove the one that is further.

If the metric is shortest path, we use Dijkstra algorithm to compute the shortest path distance to all the nodes from our nearest node. We then do the same method as above to find the  $k$  closest nodes.

We then return this as an array after reversing the order.

## 6.2 Phase 2

### 6.2.1 K shortest Paths (Exact)

For this query, we want to return the  $k$  shortest loopless paths, we use a method similar to Yen's algorithm. We first generate the shortest path, and iteratively construct alternatives by deviating from our previously found path at spur points, and temporarily avoiding edges that will reproduce our found paths, by avoiding the same prefix.

We first compute the shortest path from source to target using our already existing shortest path from Phase 1, if this does not exist then we cannot find  $k$ -shortest paths. The shortest path that we find will be our first entry in our answer list. We also use hash values for paths using a function to avoid duplicates.

Now to generate the next path, we go to the previous best path, and takes each node on that path as a spur node. We take the prefix of the previous best path till the spur node and temporarily remove edges from this node that can recreate a root prefix already in an earlier solution. Now we again compute the shortest path from the spur node to our destination, and if it exists we append it to the prefix, and form a candidate path. These candidate paths are stored in a priority queue ordered by path length. We also ensure that the edge that was blocked is restored before the next spur point.

We continue doing this for all spur points of the previous best path and take the shortest candidate from our priority queue and this will now be our next best path. This process is repeated until we get  $k$  paths, or until we can get no more unique paths, whichever happens faster. We then return the list of  $k$  paths, or lesser than  $k$  if that is the size of our answer

### 6.2.2 K Shortest Paths (Heuristic)

The heuristic method first finds the actual shortest path from the source to the destination using the normal shortest-path function. This becomes the base path and is always taken as the first (and best) answer. After that, the algorithm tries to generate different alternatives by deliberately removing small consecutive stretches (1 to  $B$  edges) from the base path at intervals of a few nodes. Every time a segment is removed, the shortest path is recomputed, which forces the search to take a different route. If the resulting path is valid and not seen before, it is added as a candidate.

For every such candidate, we compute a hash and store the path along with its length and a penalty score. The penalty is based on two things: how much the new path overlaps with the base path (compared to an overlap threshold), and how far the path’s length is from the true shortest path. After generating these candidates, we sort them by penalty and then by length, and start picking them one by one (skipping duplicates). For each chosen path, the algorithm also tries some “local spur” modifications small, local edge removals along that path to explore additional alternatives. This continues until we collect  $k$  paths or run out of candidates. Finally, all selected paths are sorted by actual length and returned.

The parameters like `psblts_cap`, `spurinterval`, and `B` help control how much diversity to aim for and how much time the method spends searching.

### 6.2.3 Approximate Shortest Paths

The approximate shortest-path module uses an ALT-style  $A^*$  search, which combines the usual  $A^*$  heuristic with additional landmarks to guide the search more aggressively. First, the system precomputes a small set of “landmark” nodes. The landmarks are selected using an avoid-based farthest-point strategy: the first landmark is chosen randomly, and each subsequent landmark is the node that maximizes its minimum distance to all previously chosen ones. This helps spread the landmarks across the graph.

During this precomputation step, the algorithm runs Dijkstra twice for each landmark—once on the forward graph and once on the reversed graph, and stores both distance tables. These tables are later used to compute tighter lower-bound estimates.

For answering approximate queries, the algorithm uses a modified  $A^*$  search. Instead of relying only on the Euclidean heuristic, it adds landmark-based bounds using the triangle inequality. For any landmark  $L$ , the algorithm can compute admissible lower bounds like

$$dist(v, t) \geq |dist(L, t) - dist(L, v)|,$$

and it takes the maximum of all such bounds across all selected landmarks. This often provides a much stronger estimate than Euclidean distance, causing  $A^*$  to explore far fewer nodes.

A weight parameter  $w$  is also applied to the heuristic, allowing a trade-off between accuracy and speed. When  $w > 1$ , the heuristic becomes more aggressive, which reduces

search time at the cost of introducing approximation.

## 7 Phase 3: Delivery Scheduling

In this phase, our goal was to build a working delivery scheduler. All delivery boys start at a depot, and we have several orders, each with a pickup and a dropoff. We needed to assign orders to drivers, find routes, and calculate delivery times. While working on this, we tried many ideas, some of which failed because of bugs or unexpected issues. This section explains what we tried, why some things did not work, and what final method we used.

### 7.1 Our Initial Plan

At the start, we planned to extend our Phase 1 and Phase 2 code. The idea was to:

- use average time on edges instead of distance,
- build a time-based shortest path function,
- add caching so repeated lookups would be faster,
- use a greedy scheduler that always picks the nearest next task.

The plan looked simple on paper, but the time-based shortest path turned out to be much harder to get working.

### 7.2 Attempt 1: Time-Based Dijkstra

We wrote a new function that tried to compute the fastest path (using `avg_time`). We also added caching so we would not recompute paths again and again.

#### Issue

This version always returned `INF` for almost every pair of nodes, even when the nodes were clearly connected. Because of this, our scheduler thought that no pickup or dropoff was reachable, so it stopped immediately and assigned nothing. After some debugging, we realised that some edge values (like `length`) were not correctly set, which broke the whole pathfinding system.

### 7.3 Attempt 2: Using Landmarks (ALT)

Next, we tried using our Phase 2 landmark code to speed things up. But this also depends on correct `length` values.

#### Issue

Since the `length` field was not set during loading, all landmark distances became `INF`. This made the ALT heuristic useless and broke all shortest path calculations again. So this approach also failed.

## 7.4 Attempt 3: Greedy Nearest Task

We then tried a greedy idea: the driver with the lowest time chooses the closest pickup or dropoff next. This sounded simple and practical.

### Issue

This method still needed the time-based shortest path function from earlier, which was still giving INF. So the greedy loop again found nothing reachable and exited without doing any work.

## 7.5 Final Working Solution

After several failed attempts, we decided to switch to a simpler and more reliable strategy.

**Key Fix:** Make `length = avg_time`

While loading the graph, we set:

```
edge.length = edge.avg_time.
```

This allowed us to reuse our already working Phase 1/2 Dijkstra code without writing any new pathfinding logic. Now, shortest paths automatically became time-based, and everything worked normally.

### Simple Assignment Strategy

Instead of a complex greedy idea, we used:

$$driver\_for\_order\ i = i \bmod num\_drivers.$$

This gives:

- fair distribution of orders,
- no bugs,
- fast and predictable scheduling.

### Route Building

For each driver:

- start at the depot,
- go to each pickup in the order given,
- then go to the corresponding dropoff,
- use our working shortest path to connect all steps,
- keep track of delivery completion times.

This produced valid routes and correct delivery-time calculations.

## 7.6 Trade-offs

This final method is simple, but we accepted some trade-offs:

- it does not always give the shortest total delivery time,
- it does not reorder tasks to be optimal,
- it does not use fancy heuristics like ALT,
- it does not batch pickups efficiently.

But it has major advantages:

- it always works,
- it gives valid output for all tests,
- it is easy to understand,
- it runs well within the time limit.

## 7.7 Conclusion

Across Phase 3, we tried:

1. time-based Dijkstra,
2. landmark-based heuristics,
3. greedy nearest-task scheduling.

All of these failed because our time-based shortest path setup was unstable. Our final method—setting `length = avg_time` and using a simple round-robin assignment—worked reliably and met all the project requirements. It produced correct routes, correct timing, and ran efficiently on all inputs.

## 8 AI Use

<https://chatgpt.com/share/6921f428-17a8-8004-9a9f-1df6d45cb7f1>  
<https://chatgpt.com/share/6921f4ad-0e74-8009-a0ee-8b40e699fa24>  
<https://chatgpt.com/share/6921f733-8d98-8009-b849-54e1e81a72e5>  
<https://chatgpt.com/share/6921fadc-05c0-800b-ad3d-79e03e72db96>  
<https://chatgpt.com/share/6921fb28-6740-800d-b2c9-98d52f1a9df7>  
<https://chatgpt.com/share/6921fb49-975c-800d-b03a-ff447d77635b>